

HolisticX Infrastructure: Developer Handover & Guide

Saifeddine Doudy / ToumAI

July 31, 2025

Contents

1	Introduction & Core Principles	2
2	Project Setup & Prerequisites	2
3	Deep Dive: The <code>live/my-project</code> Directory	2
3.1	<code>main.tf</code> : The Blueprint	2
3.2	<code>variables.tf</code> : The Contract	3
3.3	<code>env/*.tfvars</code> : The Configuration Files	3
3.4	<code>locals.tf</code> : The Scratchpad	3
3.5	<code>providers.tf</code> : The Tool Configuration	3
3.6	<code>outputs.tf</code> : The Results	3
4	Managing Environments with Workspaces	3
5	Deployment Guide	3
5.1	Step 1: Create and Select Your Workspace	3
5.2	Step 2: Create Your Variable File	4
5.3	Step 3: Cost Estimation with Infracost	4
5.4	Step 4: Plan and Apply	4
5.5	Step 5: Post-Deployment DNS	4
6	Future Enhancements: Terraform Cloud	4

1 Introduction & Core Principles

This document is the primary technical guide for the HolisticX cloud infrastructure. It details the Terraform code structure, the multi-environment deployment strategy, and the underlying AWS architecture.

- **Infrastructure as Code (IaC):** All infrastructure is managed by Terraform. **Do not make manual changes in the AWS Console.**
- **Modularity:** The code is organized into reusable modules to keep it DRY (Don't Repeat Yourself).
- **Environment Isolation:** Each environment (dev, staging, prod) is a completely separate deployment managed by Terraform Workspaces.

2 Project Setup & Prerequisites

Action Required: Configure the Terraform Backend

Before you can deploy anything, you must configure the remote backend where Terraform will store its state file.

1. **Create an S3 Bucket:** Manually create a private S3 bucket in AWS. A good name is `holisticx-terraform-state-<your-account-id>`.
2. **Update Backend Configuration:** Open `live/my-project/backend.tf` and update the bucket name.
3. **Initialize Terraform:** Run `terraform init` from `live/my-project`.

3 Deep Dive: The `live/my-project` Directory

This directory is the "composition root" where the entire infrastructure is assembled. Understanding these files is key to understanding the project.

3.1 `main.tf`: The Blueprint

This is the most important file. It defines which modules to use and how to connect them. It's like the main blueprint for the entire house. It doesn't contain logic for creating individual resources; instead, it declares module blocks and passes variables to them. For example, it passes the output of the `network` module (like the VPC ID) as an input to the `ecs.fargate` module.

3.2 `variables.tf`: The Contract

This file defines all the input variables that the root module expects. Think of it as the "contract" for the infrastructure. It defines the name and type of every variable that can be passed in from a `.tfvars` file (e.g., `variable "domain_name" { type = string }`). It also sets default values for optional variables.

3.3 `env/*.tfvars`: The Configuration Files

These files provide the actual values for the variables defined in `variables.tf`. You will have one `.tfvars` file for each environment (e.g., `develop.tfvars`, `staging.tfvars`). This is where you specify the differences between environments, such as instance sizes, domain names, and feature flags.

3.4 `locals.tf`: The Scratchpad

This file is a "scratchpad" for creating intermediate, reusable variables. It takes the raw input from the `.tfvars` file and transforms it into more useful constructs. Its most important job is to create the `common_tags` block that is applied to every resource, ensuring consistent tagging for cost allocation.

3.5 `providers.tf`: The Tool Configuration

This file configures the "tools" that Terraform uses, in this case, the AWS and PostgreSQL providers. It sets the AWS region and, crucially, defines the `default_tags` block, which automatically applies the `common_tags` from `locals.tf` to every single resource the AWS provider creates.

3.6 `outputs.tf`: The Results

This file defines what information to display after a successful `terraform apply`. It's used to output important, dynamically generated values like the RDS database endpoint or the Route 53 name servers that you need for manual configuration steps.

4 Managing Environments with Workspaces

We use Terraform Workspaces to manage multiple environments. A workspace is an independent state file, allowing you to deploy a full copy of the infrastructure without affecting other environments. The logic in `locals.tf` translates the workspace name into a standardized `Environment` tag for all resources.

5 Deployment Guide

5.1 Step 1: Create and Select Your Workspace

```
# Run from live/my-project/  
cd live/my-project
```

```
terraform workspace new <your-workspace-name>
terraform workspace select <your-workspace-name>
```

5.2 Step 2: Create Your Variable File

Copy an existing `.tfvars` file and modify it. **You must provide a globally unique name for the `rds_backup_s3_bucket_name` variable.**

5.3 Step 3: Cost Estimation with Infracost

```
# Authenticate with Infracost Cloud (one-time setup)
infracost auth login

# Generate a cost breakdown
infracost breakdown --path . \
  --terraform-var-file="env/<your-var-file>.tfvars" \
  --usage-file="infracost-usage.yml"
```

5.4 Step 4: Plan and Apply

```
terraform plan -var-file="env/<your-var-file>.tfvars"
terraform apply -var-file="env/<your-var-file>.tfvars"
```

5.5 Step 5: Post-Deployment DNS

For the first deployment, you must delegate DNS from your registrar (IONOS) to AWS.

```
terraform output name_servers
```

Log in to IONOS and update the NS records for your domain to these four values.

6 Future Enhancements: Terraform Cloud

For enhanced collaboration and security, migrating to **Terraform Cloud** is the recommended next step. It provides a more robust backend, collaborative run workflows, and policy as code features.